Paper 2 Review

Summary

This paper presents a research on automating test generation with LLM support. The purpose of it is to provide a tool for helping to create software test based on bug reports. They analyzed 300 open source projects and found 28% of tests are derived from bug reports, and made a tool to help them.

They provided a novel idea of a tool that would help developers to generate tests from reports. The tool, called Libro, has bug reports as an input and will use LLM to generate tests, process results and propose solutions when enough confidence is achieved. It suggests that the tool can generate tests from the 33% of the bugs provided, with 71% accuracy in reproduction attempts.

They evaluated Libro on real Java bugs from Defects4J, and on GitHub bugs reported after the LLM training cutoff, to check that the LLM was not just remembering the answers. That tool was compared with EvoCrash that uses code snippets in the report, measuring how often a correct test is on the top n, and how many wrong tests needs to be inspected.

Libro reproduced 91 more bugs than EvoCrash, including non-crash bugs that prior tools can't handle, ranked a correct test first for 149 bugs, and kept a similar rate (32%) on the new dataset.

The takeaway is that LLMs can turn general bug reports into tests, but the post-processing, selection and ranking steps are what make the results reliable enough for developers to trust.

Strengths

- It analyzes crash reproduction but goes beyond, as their tool was able to reproduce more bugs than EvoCrash.
- It also focuses on when to trust the LLM, rather than just counting how many bugs it reproduces.
- It is practical to run, taking about 444 seconds to generate and process 50 tests, which is within the industry standard 10 minute budget.

Weaknesses

- Their tool seems to struggle with real test infrastructure, seems to be not that useful in large projects with custom tests.
- The check against memorization is small, with only 31 new bugs, so it's hard to be sure the 33% result on Defects4J isn't helped by Codex having seen those projects in training.
- It was only evaluated on Java projects with JUnit tests, so it's unclear if it works for other languages or test styles.

Comments

Novelty

- It introduces a the problem of classifiers in reports without generating the tes.
- The LLM part is standard but they contributing on the post-processing, selection and ranking.

- Using the agreement between many LLM outputs (tests that fail with the same error) as a confidence signal is a simple but original idea for deciding when to show results to a developer.

Rigor

- The evaluation is well designed solving three research questions, with two datasets.
- Only one LLM is tested, so generalization cannot be quite proven as other models might behave differently, more models would be more convincing (especially switching frontier/open source models).
- The memorization check only looks for generated tests that exactly match the developer tests. A similarity measure instead of exact matching would make this check stronger.

Relevance

- They studied 300 real repositories and shows how the problem is relevant and important in software engineering.
- The tools evaluated are in fact industry standard and their provided solutions fills a existing gap in the testing area.
- Their analysis on 28% of tests being added from reports helps a lot in determine the relevance of a creating a tool to facilitate that process.

Verifiability and Transparency

- The paper gives a replication package and setup details, which helps in order check the work.
- The model (Codex) is now deprecated, so the experiments can't be re-run as described. Publishing all the generated tests would let others verify the results instead of trusting the generated ones at the point.
- Results are reported per project (Tables IV and VII) along with a manual failure analysis, so we can see where the tool works and where it doesn't.

Presentation

- Graphs choice is not good and not easy to follow what they try to prove.
- Algorithm snippets are hard to read and some of them might be irrelevant for the point of the discussion,
- Research questions are clearly stated and easy to read the results from it.